JETBRAINS

Your AI Agent In Any Editor

Anna Zhdan
Principal Engineer

What to expect

- Agent Client Protocol
- ACP’s design approach
- Agent integrations

Demo time!

Back to summer 2025

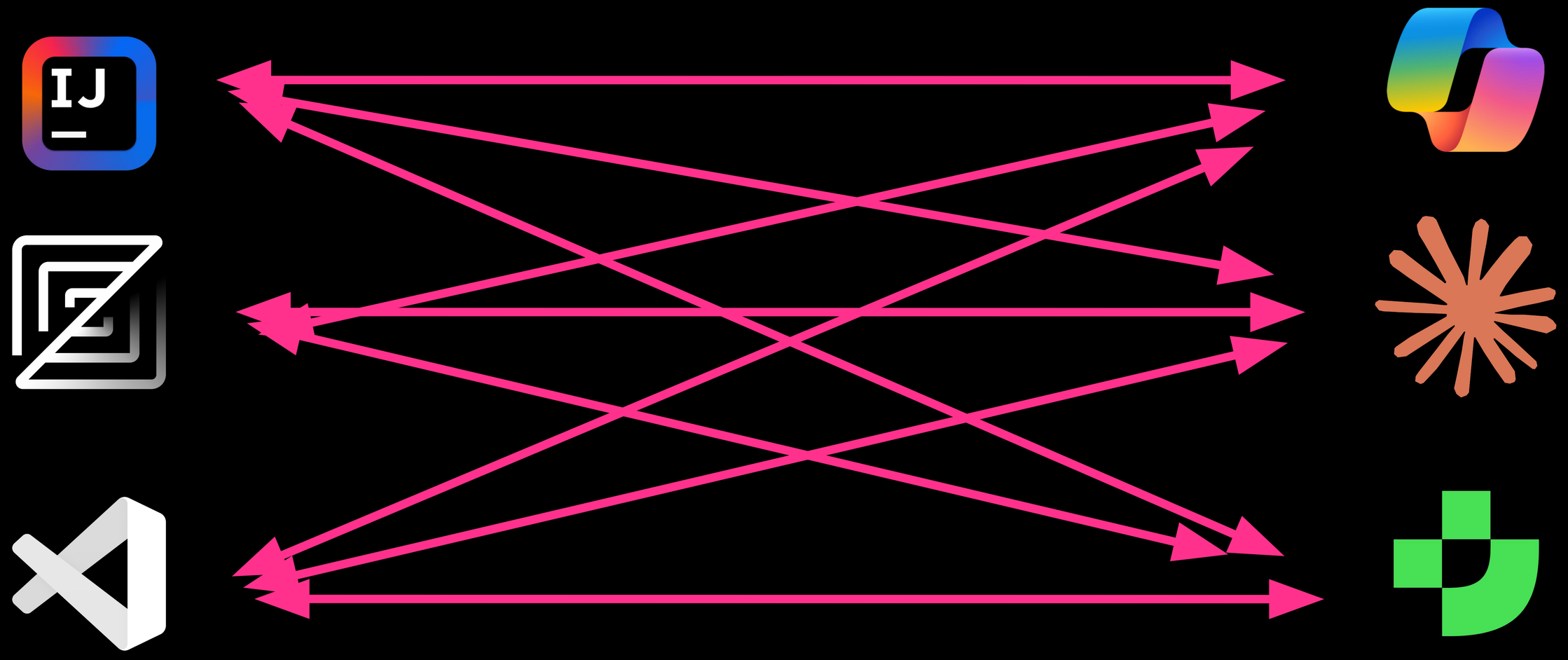
How things looked like

IDEs

AI Agents

Custom Plugins

The perfect scenario



What to do?

Invent a protocol!

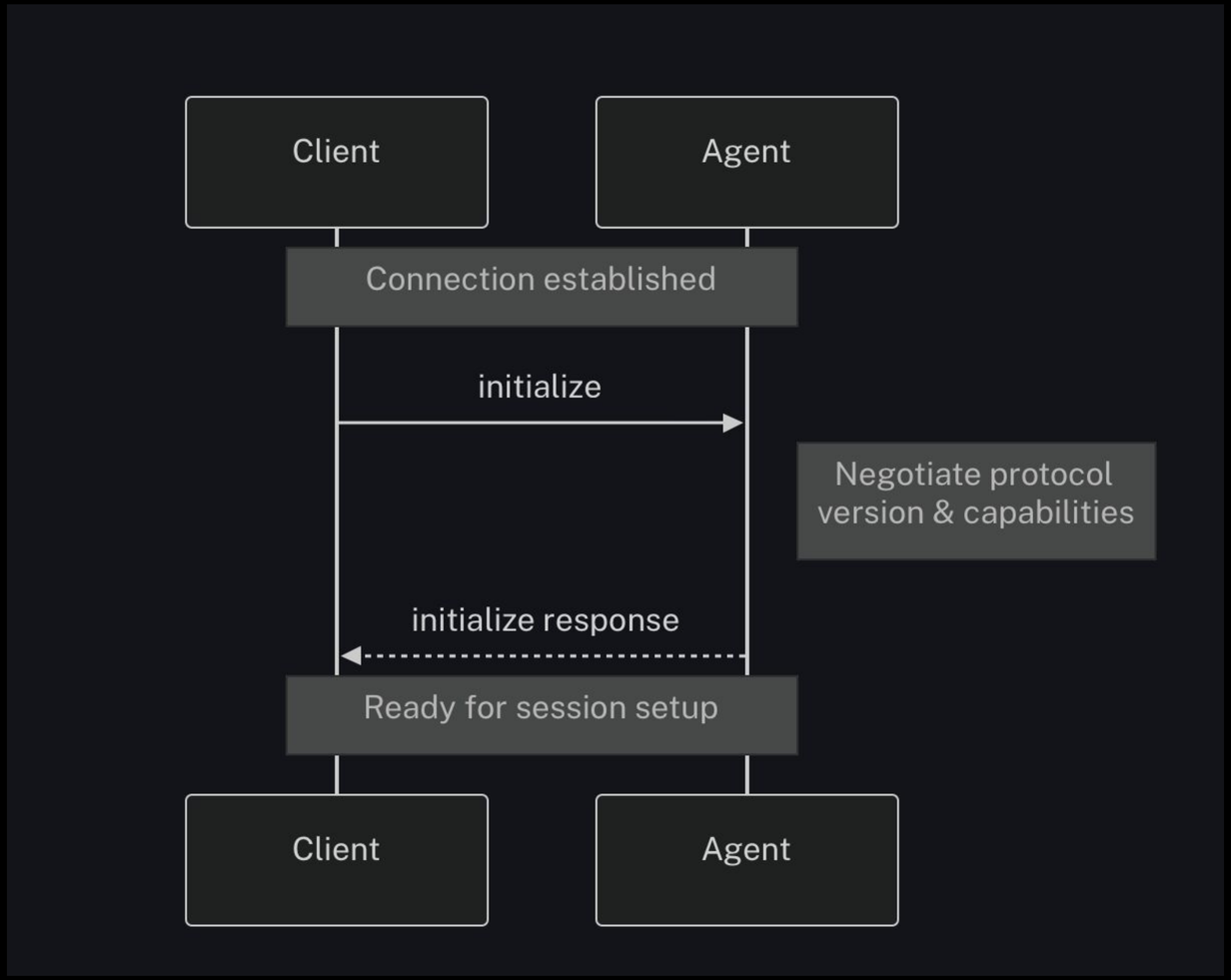
Agent/Client Protocol

Agent Client Protocol

- JSON RPC
- Stdio
- Client starts agent process and then communicates with it
- WebSocket for remote

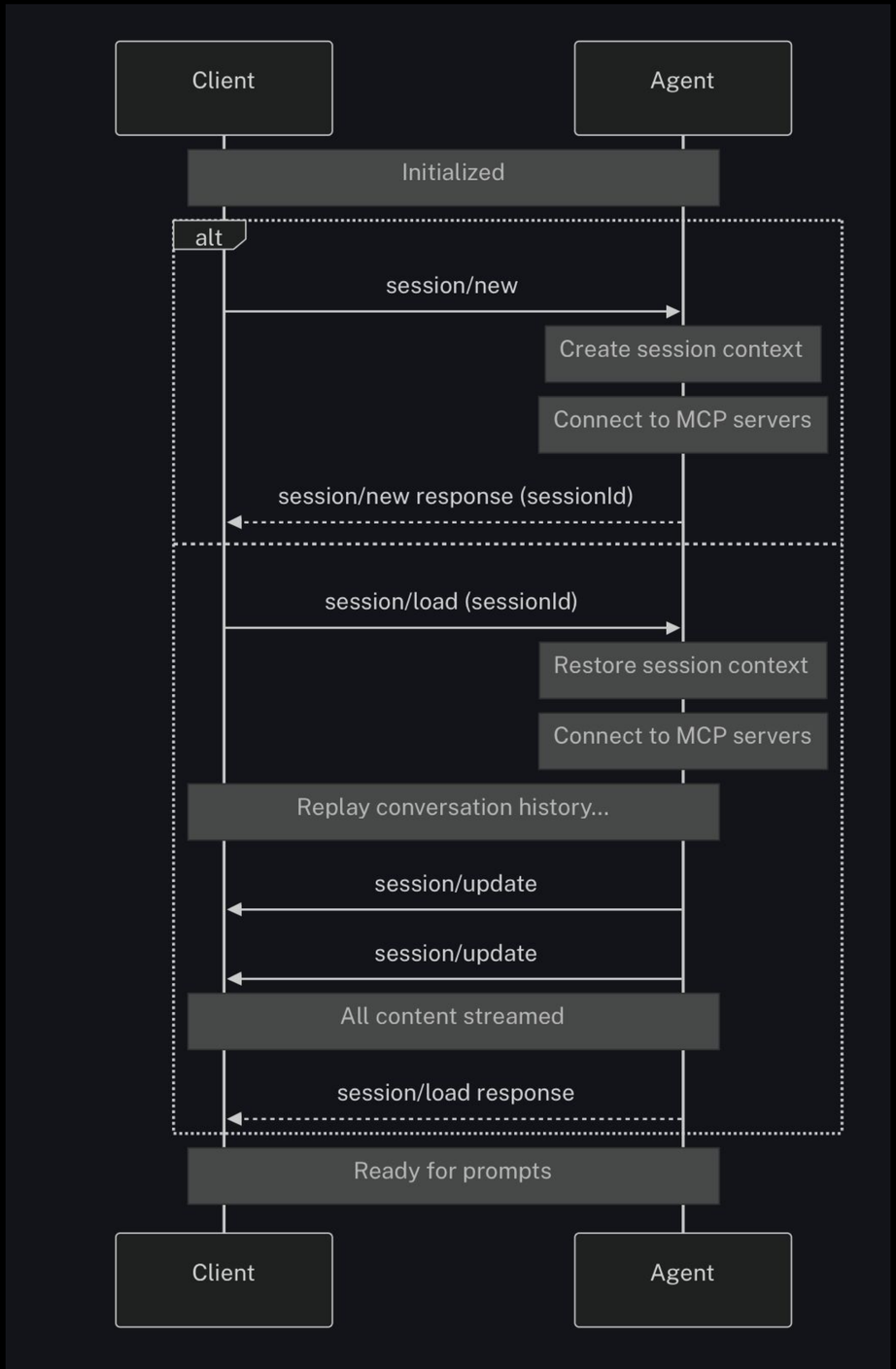
ACP: initialization

- Establish connection
- Agree on capabilities
- Get authentication methods



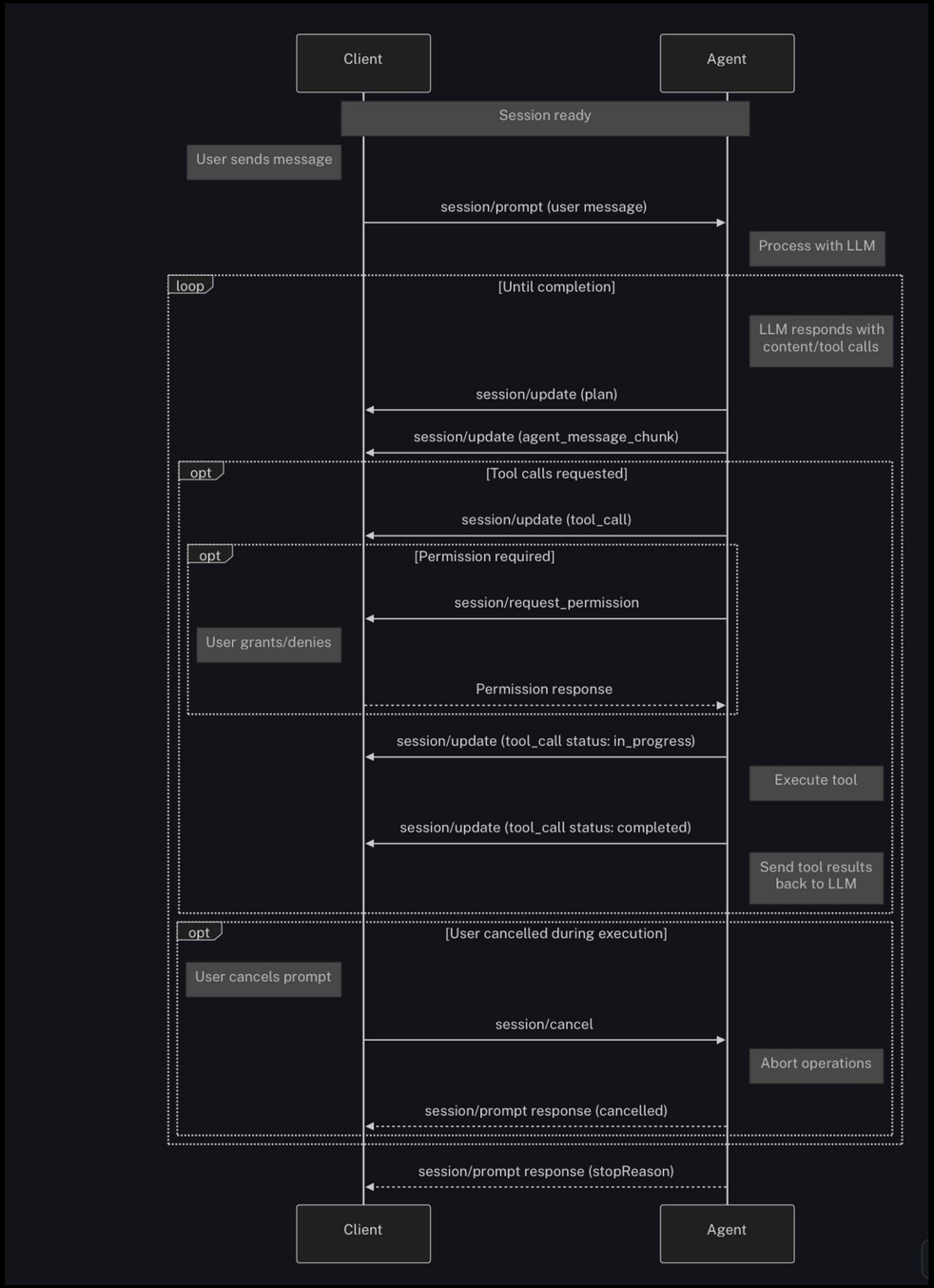
ACP: session setup

- Setup MCP
- Setup mode and model
- Get available commands



ACP: prompt turn

- Client sends prompt
- Agent streams events (tool calls, file diffs, ...)
- Agent requests permissions
- Agents returns a response to the original prompt call
- User can stop prompt execution at any time



Agents

goose

OpenCode

Gemini CLI

Codex
(via adapter)

Claude Code
(via adapter)

Junie

Kimi CLI

Clients

Zed

Emacs
Plugins

JETBRAINS

neovim

Neovim
Plugins

Obsidian
Plugins

marimo

Python Notebooks

Questions?

Choices to make: Transport

Stdio ✓
- Easier for the agents
- Less security concerns

HTTP
- Remote-native
- More natural multi-tasking

Choices to make: History

On the agent ✔

- Most agents already do that

On the client

- Agent can be stateless

Choices to make: session/load

Agent streams historical events ✓

- Resuming a session and streaming its events are different tasks →
- Very few agents support this

Client stores historical events

- Client can store the history in convenient format to present UI

* we'll still need this for the remote scenario

Solution: session/load + session/resume

session/load
Agent streams all previous events to the client

session/resume
Agent just continues the session

Choices to make: Transport

WebSocket ✓

- Less protocol changes

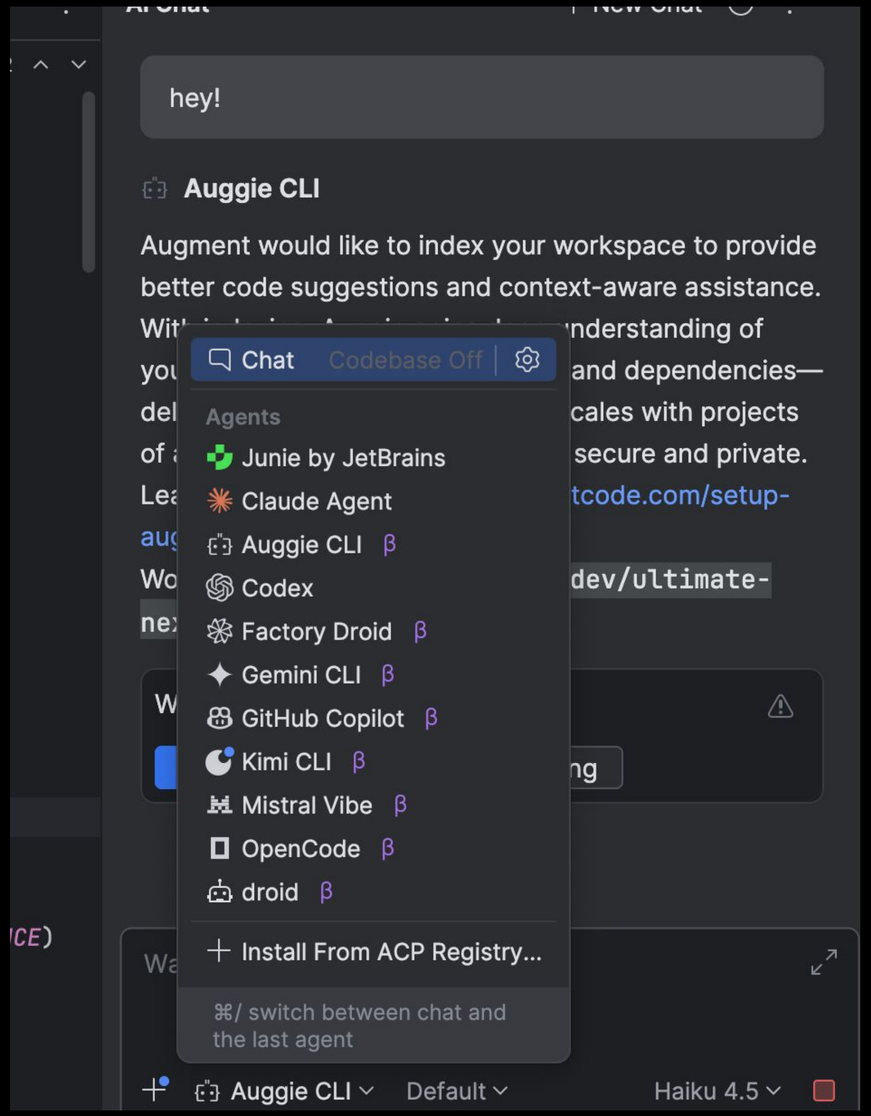
HTTP

- More top-level protocol → less manual work

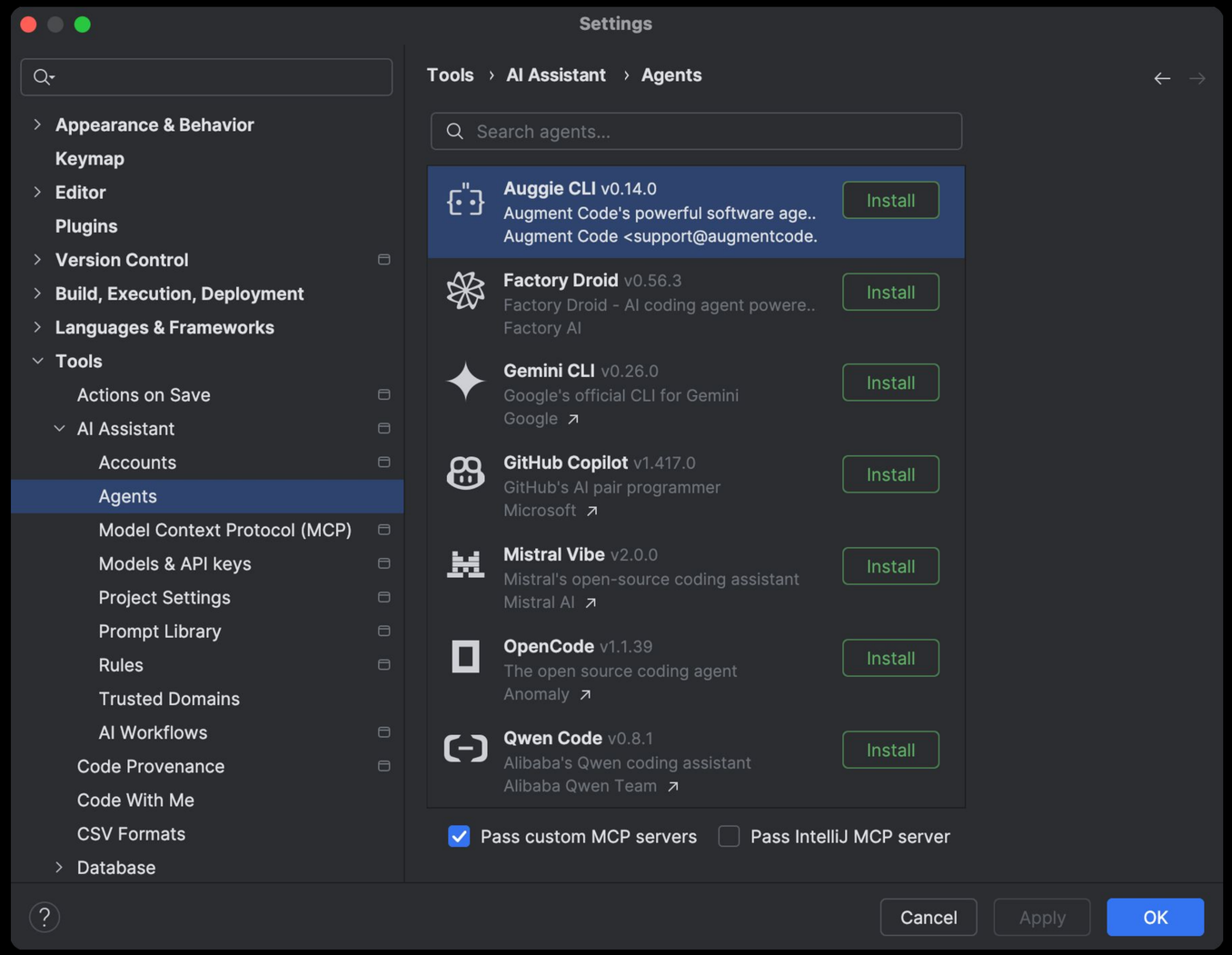
Questions?

Configuring the agent

```
{
  "agent_servers": {
    "droid": {
      "command": "droid",
      "args": ["exec", "--output-format", "acp"]
    }
  }
}
```



Agents registry



Agents registry

Just a JSON on CDN

01. Declare a list of binaries for different OSs
02. Download matching binary
03. Run the agent

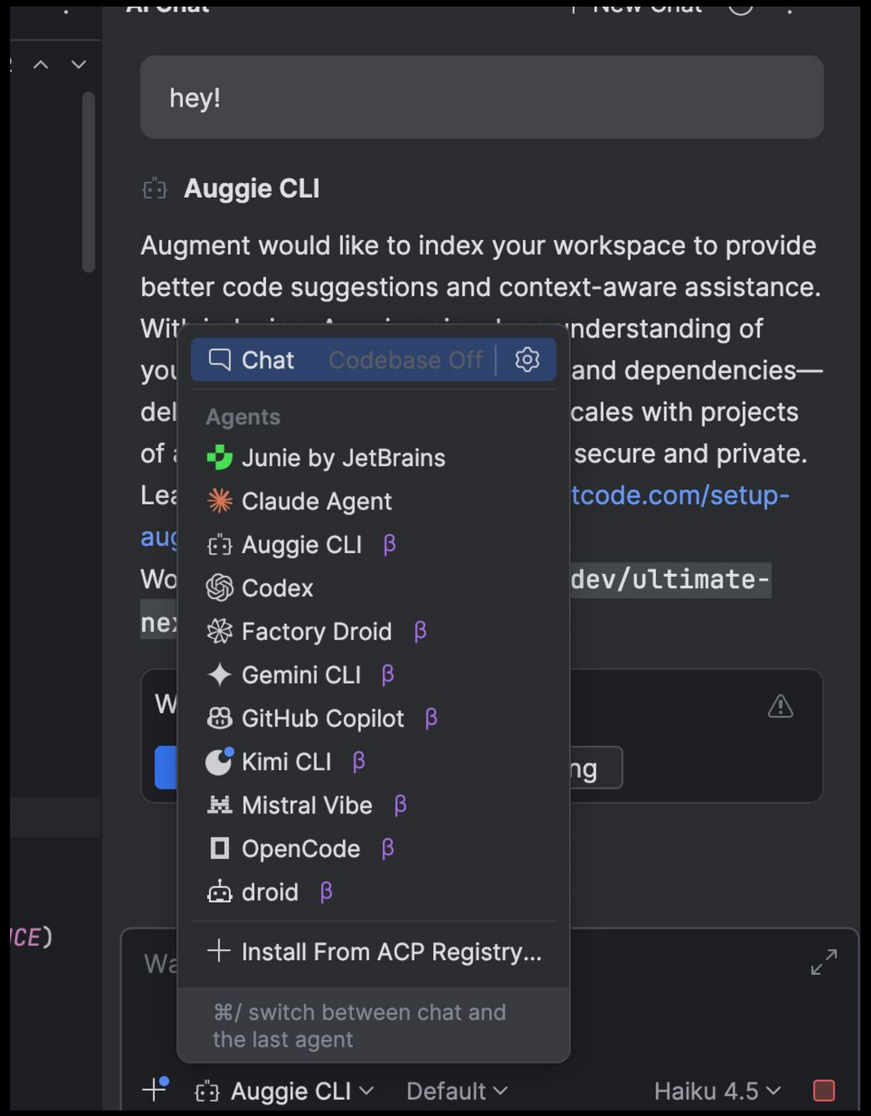
Agents registry: Node.js

01. Check that Node is installed
02. If not → download Node, setup managed runtime
03. Run the package via npx

Questions?

ACP: user’s perspective

01. Pick agents from the registry
02. Use them in your AI Chat in the IDE



ACP: agent author’s perspective

01. Implement ACP support
02. Publish agent as npm package or binary
03. Send PR to the https://github.com/agentclientprotocol/registry

Bonus: if you’re using Koog for building agents – it integrates nicely with ACP sdk

ACP: enterprise’s perspective

01. Setup custom registry URL via IDE settings provisioning
02. Publish only agents allowed by the company policy
03. Add your company’s custom agents

Questions?

Integrating the agent(s)

Biggest challenges

Level of abstraction

- What does a piece of information about agent's work look like?

Integrating the agent(s)

Biggest challenges

Level of abstraction

- What does a piece of information about agent's work look like?

Context

- How much context to add to the prompt?
- Which tools to provide?

Integrating the agent(s)

Biggest challenges

Level of abstraction
- What does a piece of information about agent's work look like?

Context
- How much context to add to the prompt?
- Which tools to provide?

How do we know it works?
- Unit tests
- Integration tests
- Evals

Level of abstraction

- Keep your UI events separate from agent's events
- Tool names can change
- Tool input/output will change

Hot take: user doesn't need too much information if it's not actionable

Context

- Coding agents collect the context themselves
- We keep initial context to the minimum, expose things via tools
- Tools can be unpredictable – add new tools one by one and check metrics

How do we know it works?

Why QA for agent integrations is **hard**

- Testing integrations is hard
- Testing non-deterministic systems is hard
- Testing of rapidly developing systems is hard
- Complex infrastructure for evals
- Running an agent costs money

How do we know it works?

What worked for us

- If something can be tested deterministically – do it
- Separate tests for all the parts
- Record all the JSON RPC traffic, can replay events in tests
- Mock “agent” that just replays events for reproducing bugs

Thank you!

Anna Zhdan
anna.zhdan@jetbrains.com